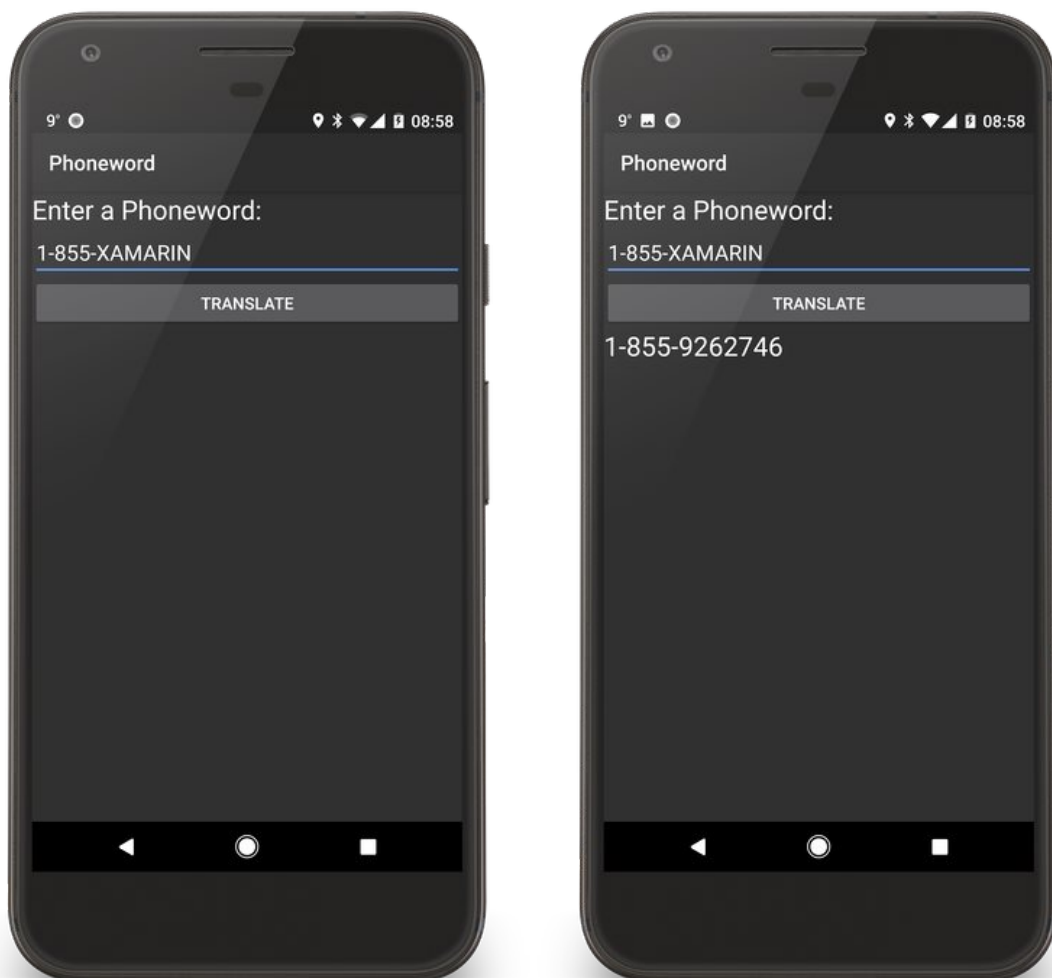


Hello, Android: Quickstart

Introduction to Android development with Xamarin

Hello, Android Quickstart

In this walkthrough, you will create an application that translates an alphanumeric phone number (entered by the user) into a numeric phone number and display the numeric phone number to the user. The final application looks like this:



Requirements

To follow along with this walkthrough, you will need the following:

[[ide name="vs"]]

- Windows 7 or later.
- Visual Studio 2015 Professional or later.

[/ide]

[[ide name="xs"]]

- The latest version of Visual Studio for Mac.
- OS X Yosemite or later.

[/ide]

This walkthrough assumes that the latest version of Xamarin.Android is installed and running on your platform of choice. For a guide to installing Xamarin.Android, refer to the [Xamarin.Android Installation](#) guides. Before you get started, please download and unzip the [Xamarin App Icons & Launch Screens](#) set.

Configuring Emulators

If you are using Google's Android SDK emulator, we recommend that you configure the emulator to use hardware acceleration. Instructions for configuring hardware acceleration are available in [Hardware Acceleration](#).

[[ide name="vs"]]

If you are using the Visual Studio Android Emulator, Hyper-V must be enabled on your computer. For more information about configuring the Visual Studio Android Emulator, see [System Requirements for the Visual Studio Emulator for Android](#).

[/ide]

[[ide name="xs"]]

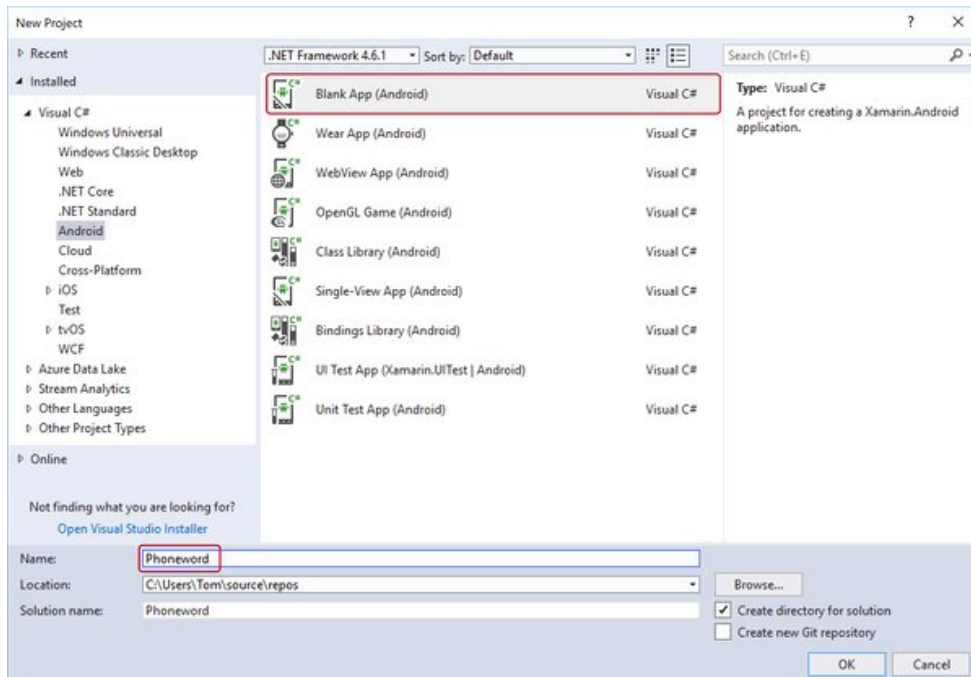
[/ide]

Walkthrough

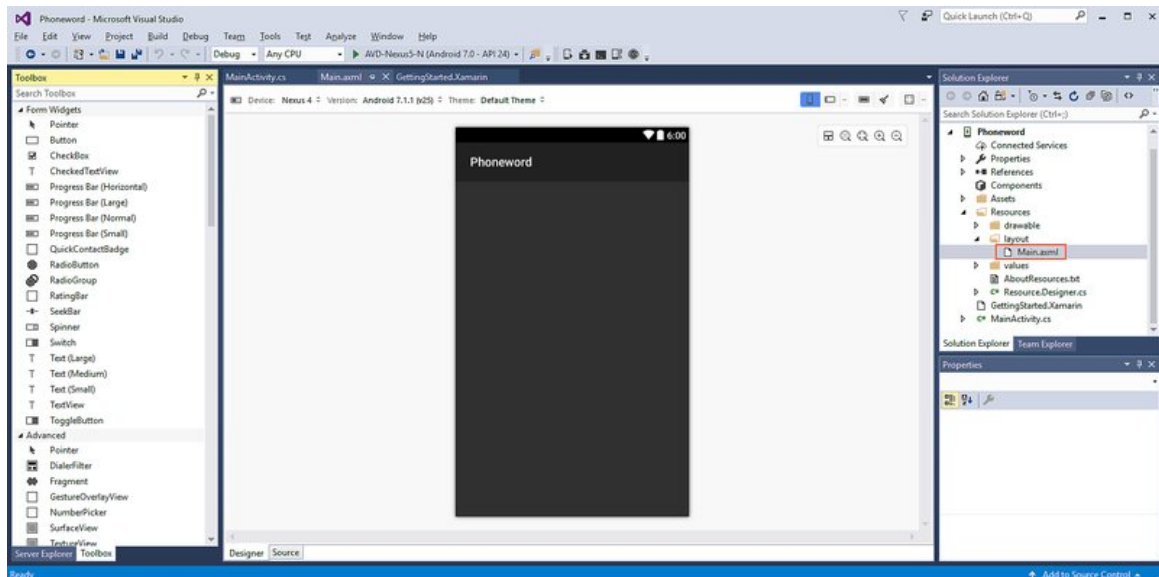
[[ide name="vs"]]

1. Start Visual Studio. Click **File > New > Project** to create a new project.

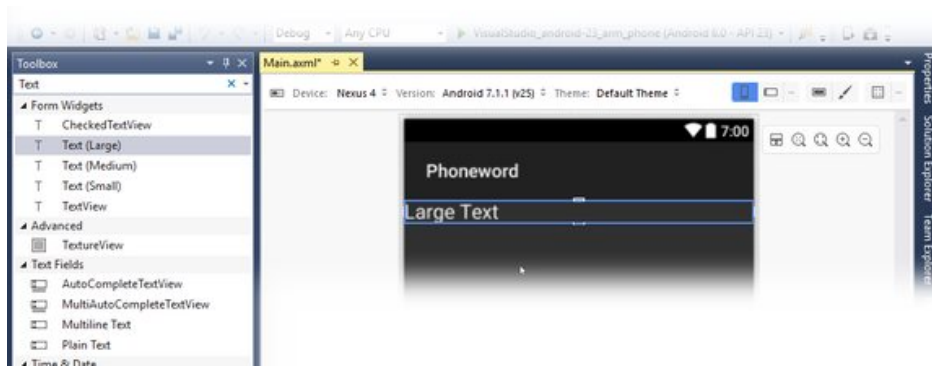
2. In the **New Project** dialog, click the **Blank App (Android)** template. Name the new project `Phoneword`. Click **OK** to create the new project:



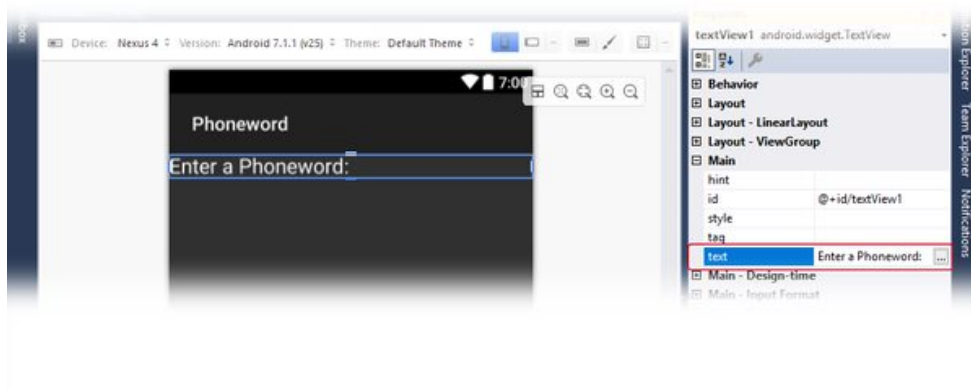
3. After the new project is created, expand the **Resources** folder and then the **layout** folder in the **Solution Explorer**. Double-click **Main.axml** to open it in the Android Designer. This is the layout file for the app's screen:



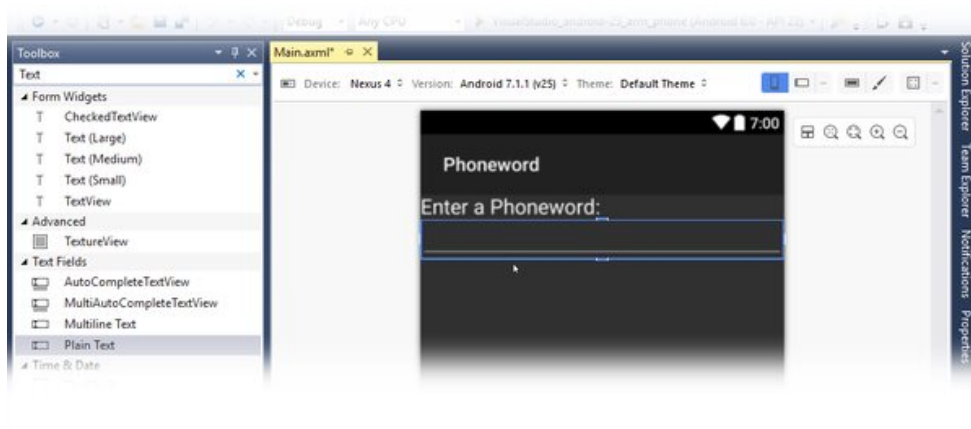
4. From the **Toolbox** (the area on the left), enter `text` into the search field and drag a **Text (Large)** widget onto the design surface (the area in the center):



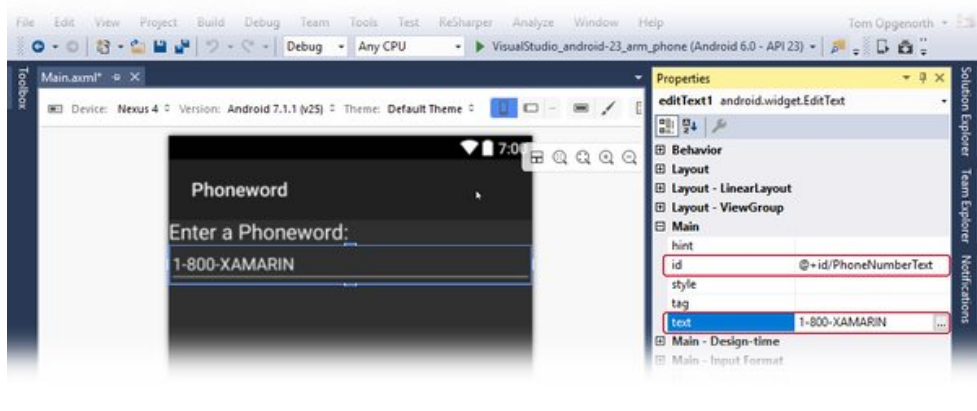
- With the **Text (Large)** control selected on the design surface, use the **Properties** pane to change the `text` property of the **Text (Large)** widget to `Enter a Phoneword:` as shown here:



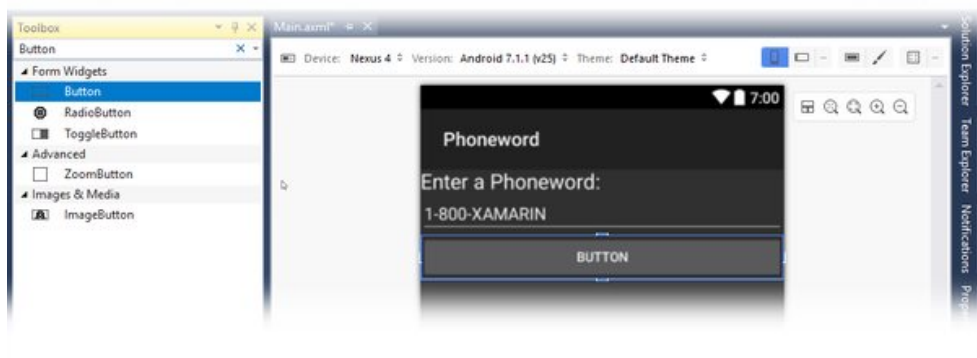
- Drag a **Plain Text** widget from the **Toolbox** to the design surface and place it underneath the **Text (Large)** widget:



- With the **Plain Text** widget selected on the design surface, use the **Properties** pane to change the `id` property of the **Plain Text** widget to `@+id/PhoneNumberText` and change the `text` property to `1-855-XAMARIN`:



8. Drag a **Button** from the **Toolbox** to the design surface and place it underneath the **Plain Text** widget:

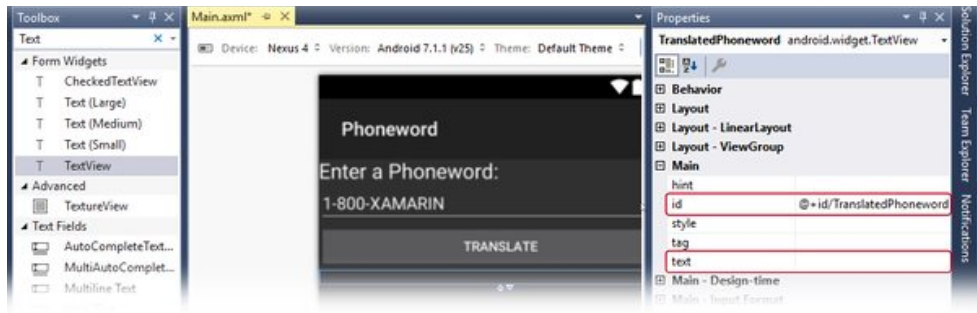


9. With the **Button** selected on the design surface, use the **Properties** pane to change the `id` property of the **Button** to `@+id/TranslateButton` and change the `text` property to `Translate`:

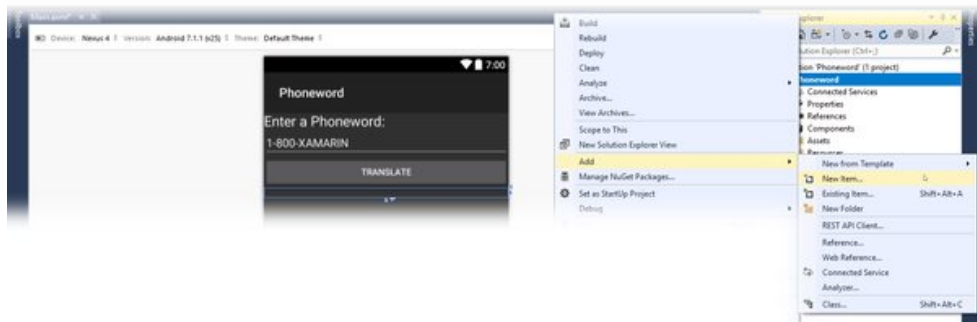


10. Drag a **TextView** from the **Toolbox** to the design surface and place it under the **Button** widget. Set the `id` property of the **TextView** to `@+id/TranslatedPhoneWord` and change the `text` to an empty string:

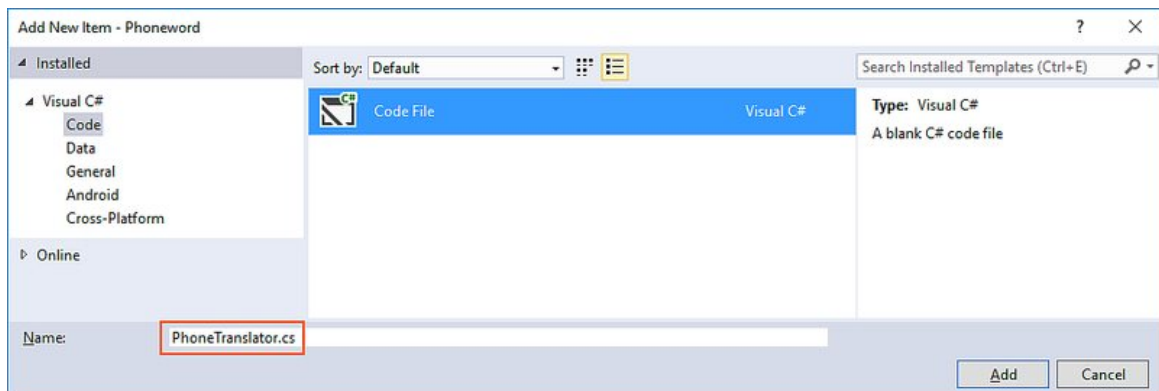




11. Save your work by pressing **CTRL+S**.
12. The next step is to add some code to translate phone numbers from alphanumeric to numeric. Add a new file to the project by right-clicking the **Phoneword** project in the **Solution Explorer** pane and choosing **Add > New Item...** as shown below:



13. In the **Add New Item** dialog, select **Visual C# > Code** and name the new code file **PhoneTranslator.cs**:



14. This creates a new empty C# class. Insert the following code into this file:

```
using System.Text;
using System;
namespace Core
{
```

```

public static class PhonewordTranslator
{
    public static string ToNumber(string raw)
    {
        if (string.IsNullOrEmpty(raw))
            return "";
        else
            raw = raw.ToUpperInvariant();

        var newNumber = new StringBuilder();
        foreach (var c in raw)
        {
            if (" -0123456789".Contains(c))
            {
                newNumber.Append(c);
            }
            else
            {
                var result = TranslateToNumber(c);
                if (result != null)
                    newNumber.Append(result);
            }
            // otherwise we've skipped a non-numeric char
        }
        return newNumber.ToString();
    }
    static bool Contains (this string keyString, char c)
    {
        return keyString.IndexOf(c) >= 0;
    }
    static int? TranslateToNumber(char c)
    {
        if ("ABC".Contains(c))
            return 2;
        else if ("DEF".Contains(c))
            return 3;
        else if ("GHI".Contains(c))
            return 4;
        else if ("JKL".Contains(c))

```

```

        return 5;
    else if ("MNO".Contains(c))
        return 6;
    else if ("PQRS".Contains(c))
        return 7;
    else if ("TUV".Contains(c))
        return 8;
    else if ("WXYZ".Contains(c))
        return 9;
    return null;
    }
}
}

```

Save the changes to the **PhoneTranslator.cs** file by clicking **File > Save** (or by pressing **CTRL+S**), then close the file.

15. The next step is to add code to wire up the user interface by inserting backing code into the `MainActivity` class. Begin by wiring up the **Translate** button. In the `MainActivity` class, find the `OnCreate` method. The next step is to add the button code inside `OnCreate`, below the `base.OnCreate(bundle)` and `SetContentView (Resource.Layout.Main)` calls. First, modify the template code so that the `OnCreate` method resembles the following:

```

using System;
using Android.App;
using Android.Content;
using Android.Widget;
using Android.OS;

namespace Phoneword
{
    [Activity (Label = "Phone Word", MainLauncher = true)]
    public class MainActivity : Activity
    {
        protected override void OnCreate (Bundle bundle)
        {
            base.OnCreate (bundle);

            // Set our view from the "main" layout resource
            SetContentView (Resource.Layout.Main);
        }
    }
}

```



```

        // New code will go here
    }
}
}

```

16. Get a reference to the controls that were created in the layout file via the Android Designer. Add the following code inside the `OnCreate` method, after the call to `SetContentView`:

```

// Get our UI controls from the loaded layout
EditText phoneNumberText = FindViewById<EditText>(Resource.Id.PhoneNumberText);
TextView translatedPhoneWord = FindViewById<TextView>
(Resource.Id.TranslatedPhoneWord);
Button translateButton = FindViewById<Button>(Resource.Id.TranslateButton);

```

17. Add code that responds to user presses of the **Translate** button. Add the following code to the `OnCreate` method (after the lines added in the previous step):

```

// Add code to translate number
translateButton.Click += (sender, e) =>
{
    // Translate user's alphanumeric phone number to numeric
    string translatedNumber =
Core.PhonewordTranslator.ToNumber(phoneNumberText.Text);
    if (string.IsNullOrEmpty(translatedNumber))
    {
        translatedPhoneWord.Text = string.Empty;
    }
    else
    {
        translatedPhoneWord.Text = translatedNumber;
    }
};

```

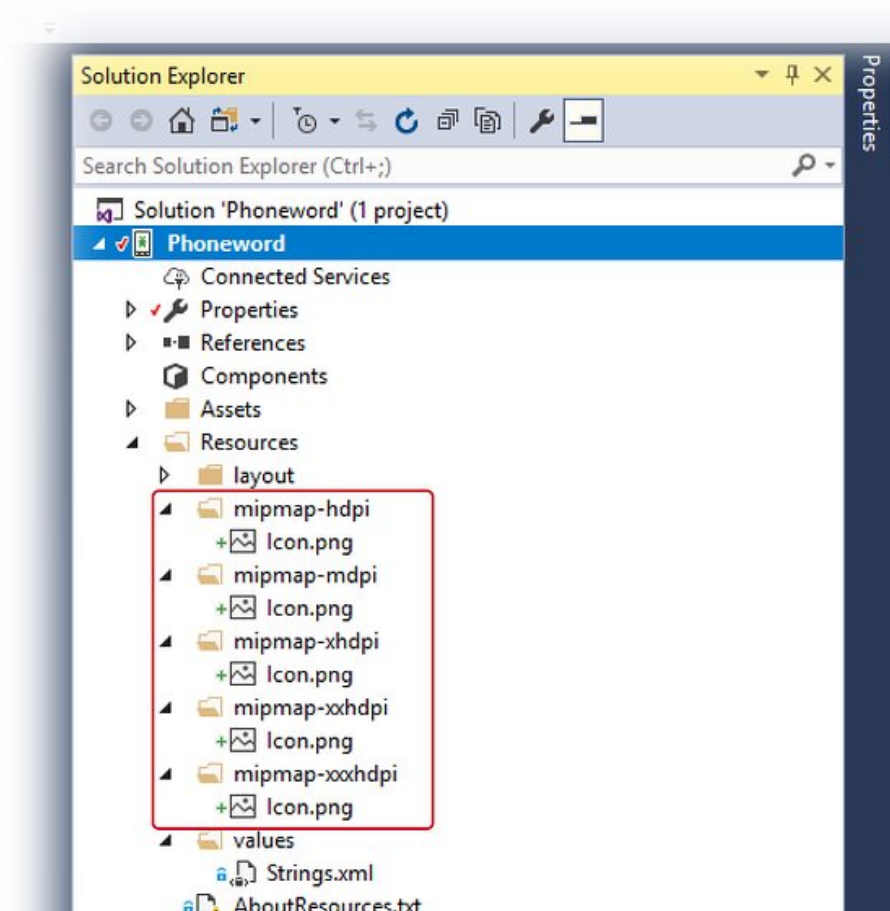
18. Save your work by selecting **File > Save All** (or by pressing **CTRL-SHIFT-S**) and build the application by selecting **Build > Rebuild Solution** (or by pressing **CTRL-SHIFT-B**).

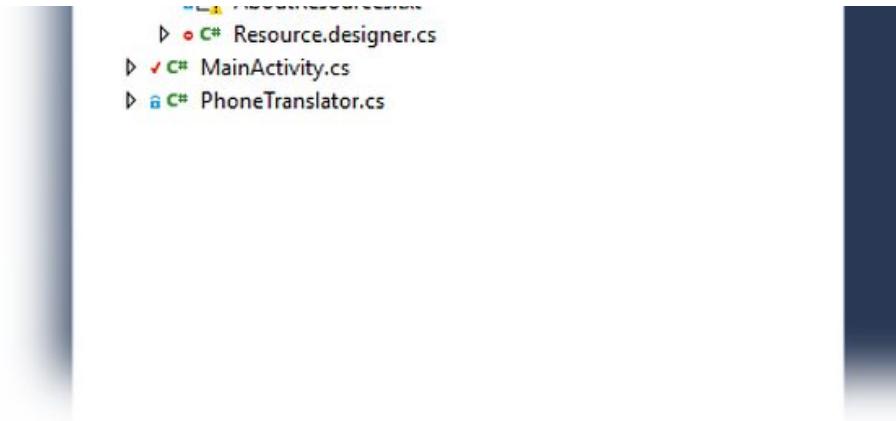
If there are errors, go through the previous steps and correct any mistakes until the application builds successfully. If you get a build error such as, *Resource does not exist in the current context*, verify that the namespace name in **MainActivity.cs** matches the project name (**Phoneword**) and then completely rebuild the solution. If you still get build errors, verify that you have installed the latest Xamarin.Android updates.

19. You should now have a working application – it's time to add the finishing touches! In **MainActivity.cs**, edit the `Label` for the `MainActivity`. The `Label` is what Android displays at the top of the screen to let users know where they are in the application. At the top of the `MainActivity` class, change the `Label` to `Phone Word` as shown here:

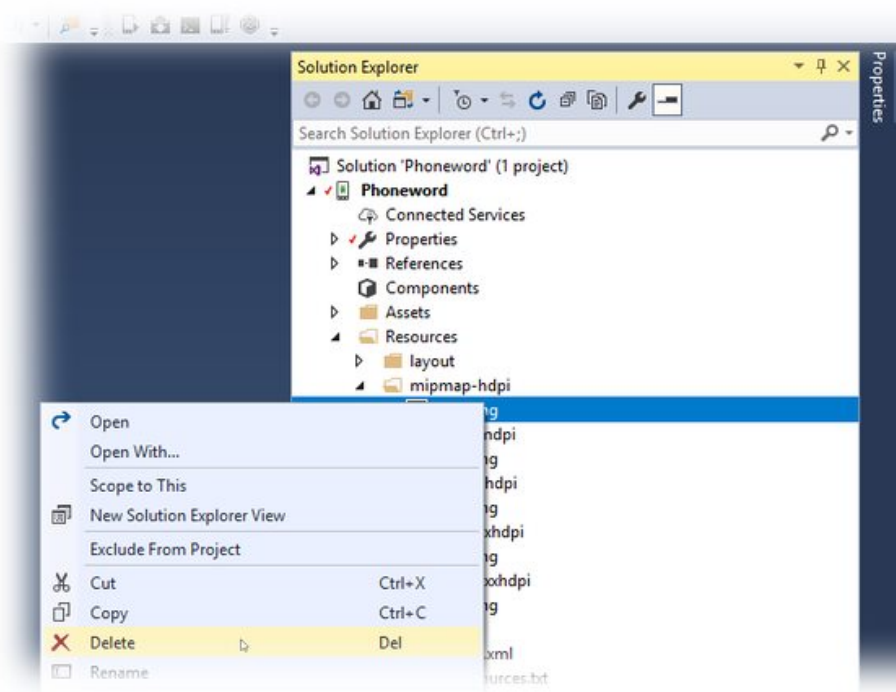
```
namespace Phoneword
{
    [Activity (Label = "Phone Word", MainLauncher = true)]
    public class MainActivity : Activity
    {
        ...
    }
}
```

20. Now it's time to set the application icon. By default, Visual Studio will provide a default icon for the project. Let's delete these files from the solution, and replace them with a different icon. Expand the **Resources** folder in the **Solution Pad**. Notice that there are five folders that are prefixed with **mipmap-**, and that each of these folders contains a single **Icon.png** file:



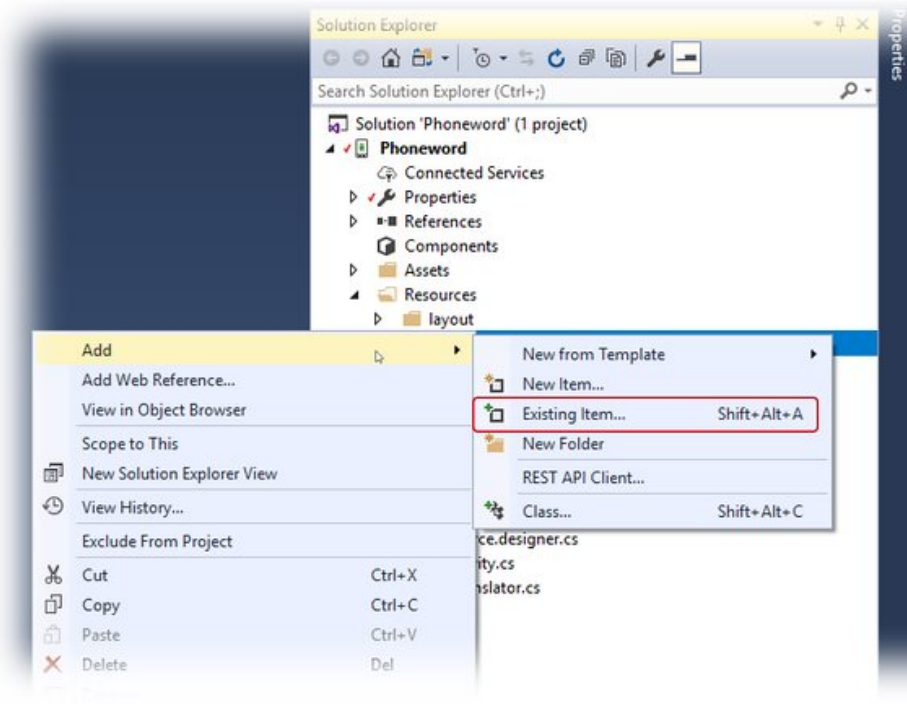


It is necessary to delete each of these icon files from the project. Right click on each of **Icon.png** files, and select **Delete** from the context menu:

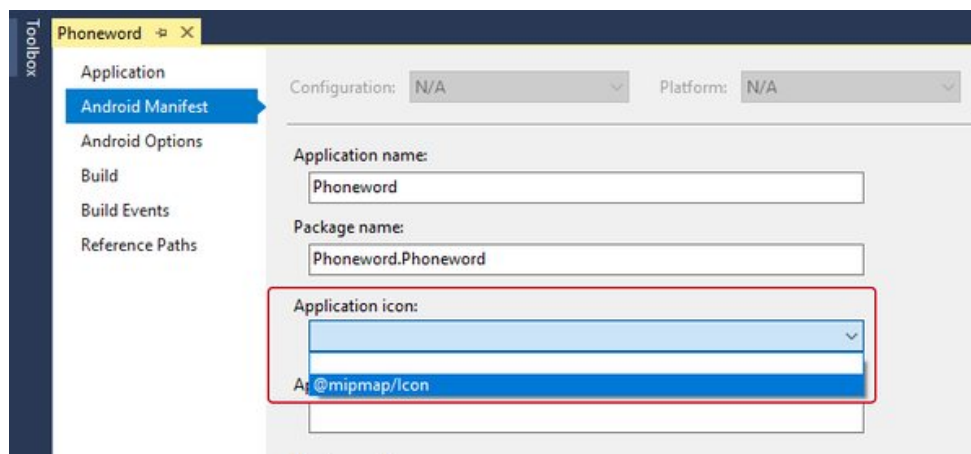


Click on the **Delete** button in the dialog.

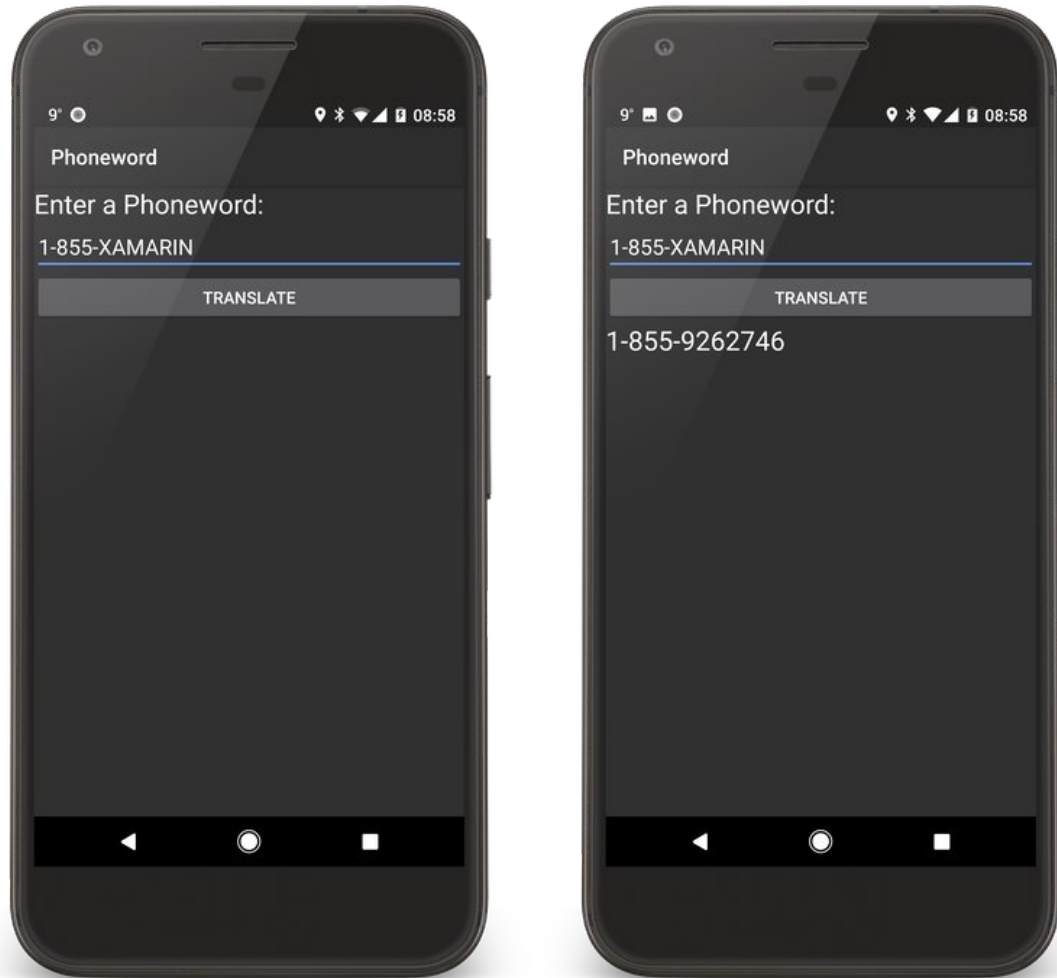
21. Next, download and unzip [Xamarin App Icons set](#). This zip file holds the icons for the application. Each icon is visually identical but at different resolutions it renders correctly on different devices with different screen densities. The set of files must be copied into the Xamarin.Android project. In Visual Studio, in the **Solution Explorer**, right-click the **mipmap-hdpi** folder and select **Add > Existing Items**:



22. From the selection dialog, navigate to the unzipped Xamarin AdApp Icons directory and open the **mipmap-hdpi** folder. Select **Icon.png** and click **Add**.
23. Repeat these steps for each of the **mipmap-** folders until the contents of the **mipmap-** Xamarin App Icons folders are copied to their counterpart **mipmap-** folders in the **Phoneword** project.
24. After all the icons are copied to the Xamarin.Android project, open the **Project Options** dialog by right clicking on the project in the **Solution Pad**. Select **Build > Android Application** and select **@mipmap/icon** from the **Application icon** combo box:



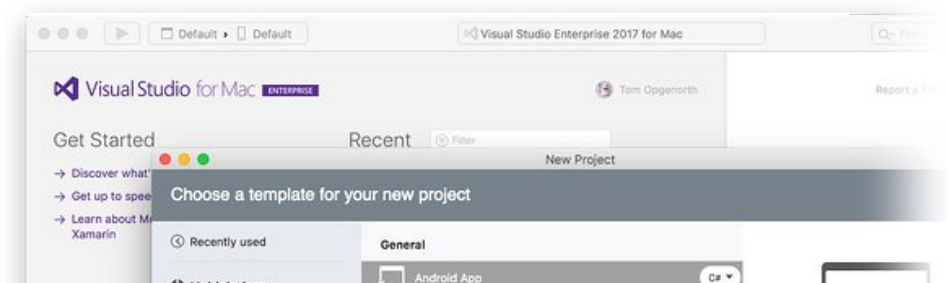
25. Finally, test the application by running it on an Android device or emulator and translating a Phoneword:

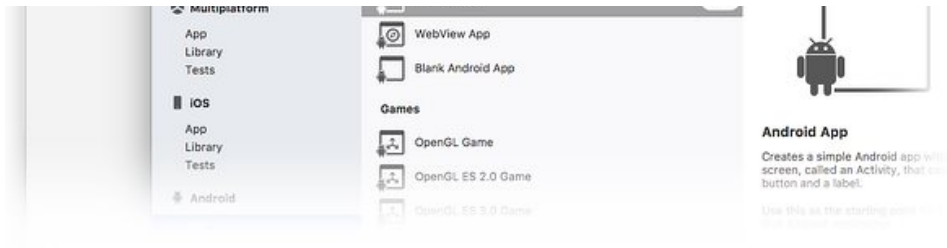


[[/ide]]

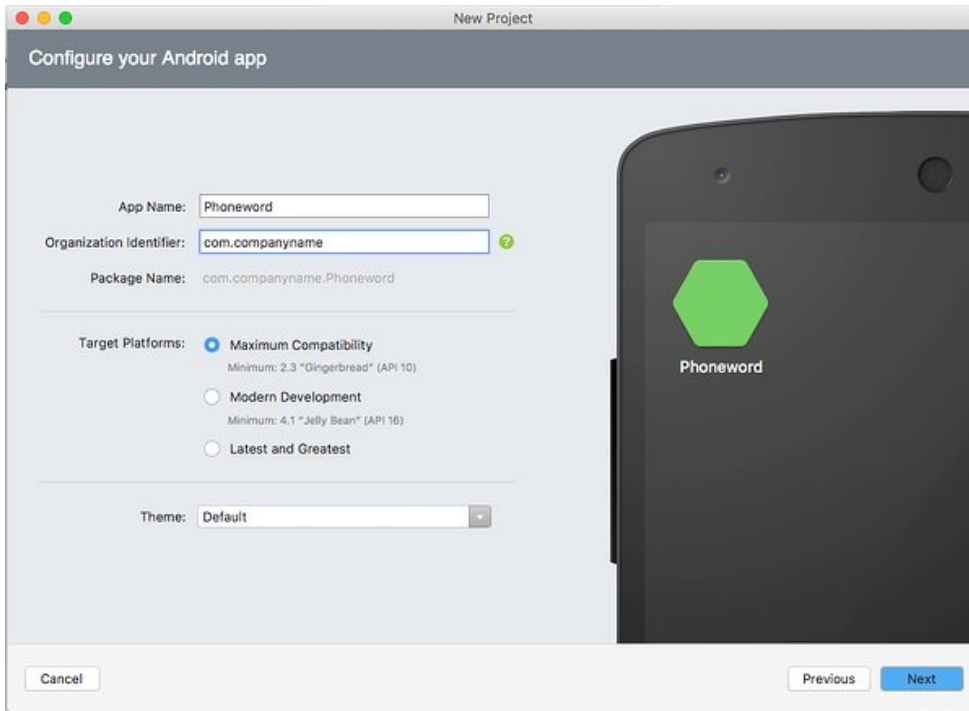
[[ide name="xs"]]

1. Launch Visual Studio for Mac from the **Applications** folder or from **Spotlight**.
2. Click **New Solution...** to create a new project.
3. In the **Choose a template for your new project** dialog, click **Android > App** and select the **Android App** template. Click **Next**.



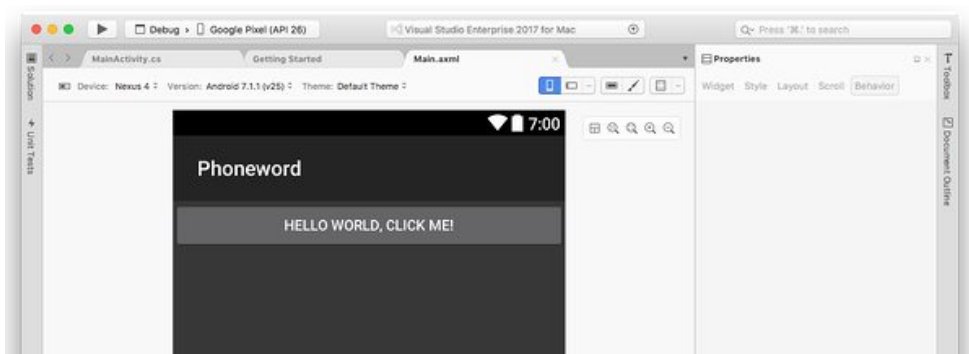


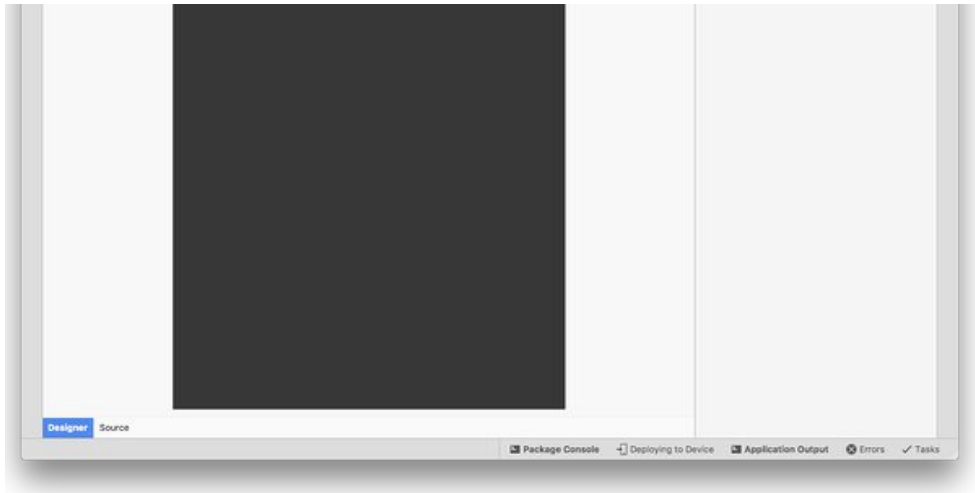
4. In the **Configure your Android app** dialog, name the new app `Phoneword` and click **Next**.



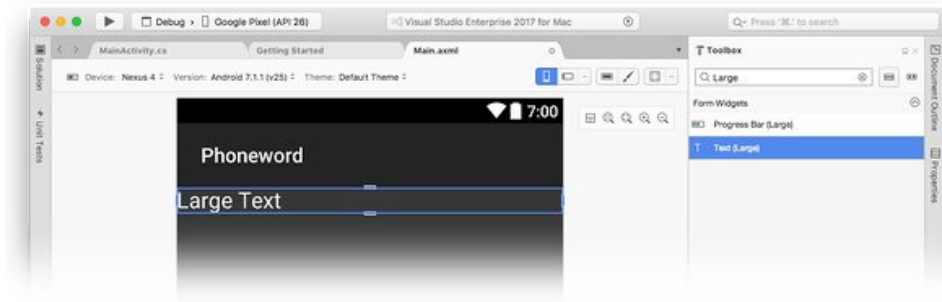
In the **Configure your new Android App** dialog, leave the Solution and Project names set to `Phoneword` and click **Create** to create the project.

5. After the new project is created, expand the **Resources** folder and then the **layout** folder in the **Solution** pad. Double-click **Main.xml** to open it in the Android Designer. This is the layout file for the screen when it is viewed in the Android Designer:

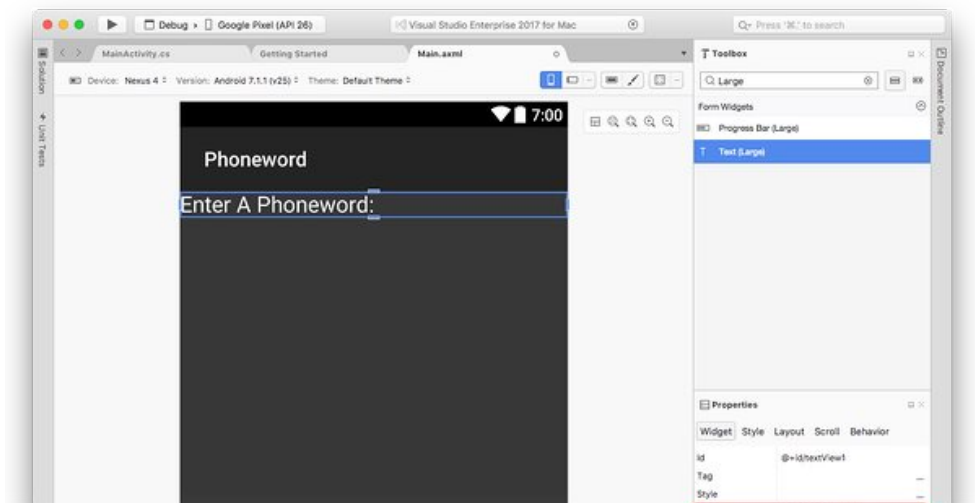




6. Select the **Hello World, Click Me! Button** on the design surface and press the **Delete** key to remove it.
7. From the **Toolbox** (the area on the right), enter `text` into the search field and drag a **Text (Large)** widget onto the design surface (the area in the center):

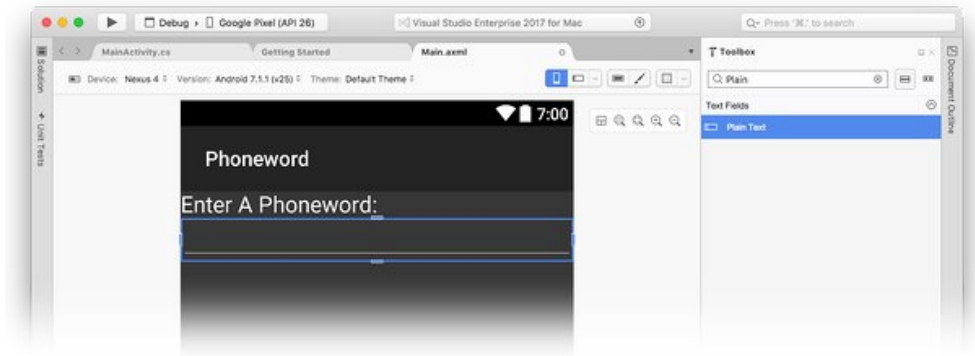


8. With the **Text (Large)** widget selected on the design surface, you can use the **Properties** pad to change the **Text** property of the **Text (Large)** widget to `Enter a Phoneword:` as shown below:

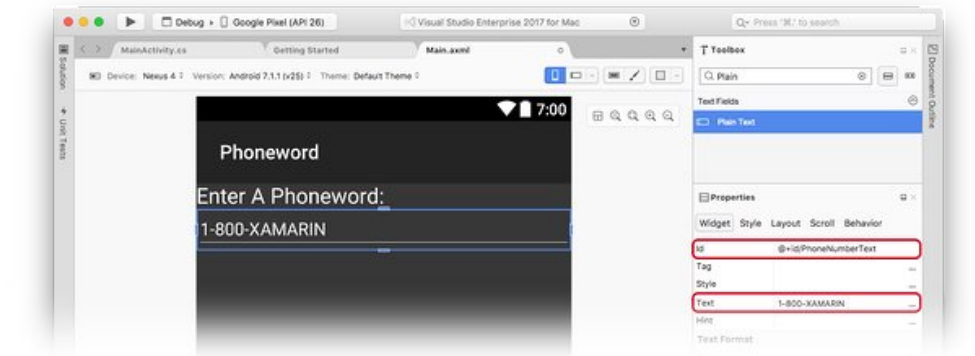




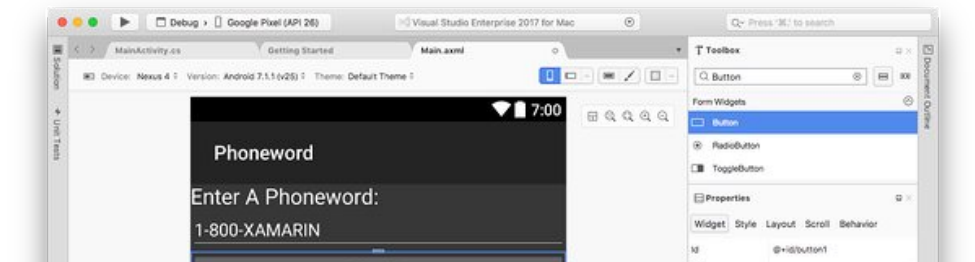
9. Next, drag a **Plain Text** widget from the **Toolbox** to the design surface and place it underneath the **Text** (Large) widget. Notice that you can use the search field to help locate widgets by name:



10. With the **Plain Text** widget selected on the design surface, you can use the **Properties** pad to change the **Id** property of the **Plain Text** widget to `@+id/PhoneNumberText` and change the **Text** property to `1-855-XAMARIN`:

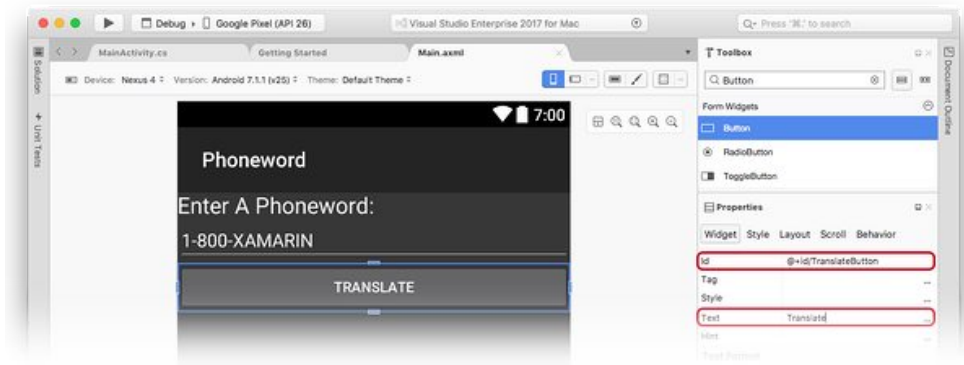


11. Drag a **Button** from the **Toolbox** to the design surface and place it underneath the **Plain Text** widget:

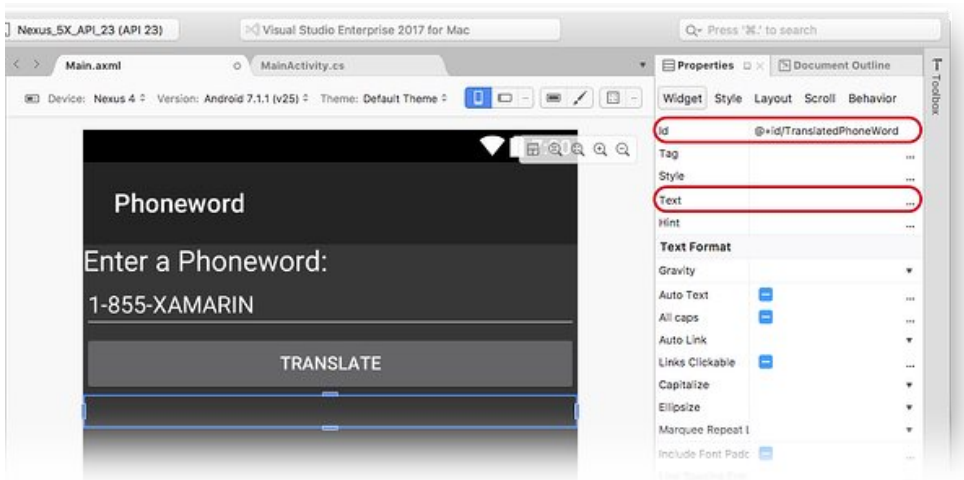




12. With the **Button** selected on the design surface, you can use the **Properties** pad to change the **Id** property of the **Button** to `@+id/TranslateButton` and change the **Text** property to `Translate`:

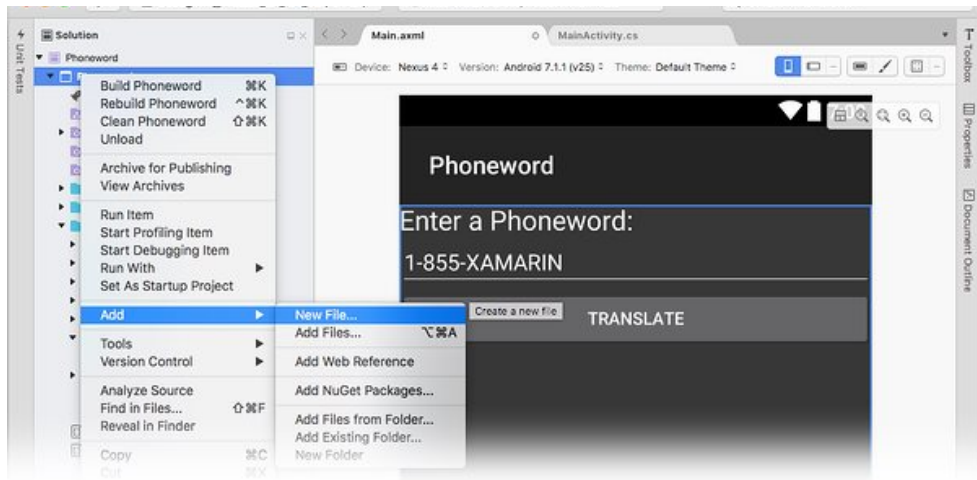


13. Drag a **TextView** from the **Toolbox** to the design surface and place it under the **Button** widget. With the **TextView** selected, set the **id** property of the **TextView** to `@+id/TranslatedPhoneWord` and change the **text** to an empty string:



14. Save your work by pressing **⌘ + S**.
15. Now, add some code to translate phone numbers from alphanumeric to numeric. Add a new file to the project by clicking the gear icon next to the **Phoneword** project in the **Solution** pad and choosing **Add > New File...**:





16. In the **New File** dialog, select **General > Empty Class**, name the new file **PhoneTranslator**, and click **New**. This creates a new empty C# class for us.
17. Remove all of the template code in the new class and replace it with the following code:

```
using System.Text;
using System;
namespace Core
{
    public static class PhonewordTranslator
    {
        public static string ToNumber(string raw)
        {
            if (string.IsNullOrEmpty(raw))
                return "";
            else
                raw = raw.ToUpperInvariant();

            var newNumber = new StringBuilder();
            foreach (var c in raw)
            {
                if ("0123456789".Contains(c))
                {
                    newNumber.Append(c);
                }
                else
                {

```

```

        var result = TranslateToNumber(c);
        if (result != null)
            newNumber.Append(result);
    }
    // otherwise we've skipped a non-numeric char
}
return newNumber.ToString();
}
static bool Contains (this string keyString, char c)
{
    return keyString.IndexOf(c) >= 0;
}
static int? TranslateToNumber(char c)
{
    if ("ABC".Contains(c))
        return 2;
    else if ("DEF".Contains(c))
        return 3;
    else if ("GHI".Contains(c))
        return 4;
    else if ("JKL".Contains(c))
        return 5;
    else if ("MNO".Contains(c))
        return 6;
    else if ("PQRS".Contains(c))
        return 7;
    else if ("TUV".Contains(c))
        return 8;
    else if ("WXYZ".Contains(c))
        return 9;
    return null;
}
}
}

```

Save the changes to the **PhoneTranslator.cs** file by choosing **File > Save** (or by pressing **⌘ + S**), then close the file. Ensure that there are no compile-time errors by rebuilding the solution.

18. The next step is to add code to wire up the user interface by adding the backing code into the `MainActivity` class. Double-click **MainActivity.cs** in the **Solution Pad** to open it.

19. Begin by adding an event handler to the **Translate** button. In the `MainActivity` class, find the `OnCreate` method. Add the button code inside `OnCreate`, below the `base.OnCreate(bundle)` and `SetContentView (Resource.Layout.Main)` calls. Remove the template button handling code so that the `OnCreate` method resembles the following:

```
using System;
using Android.App;
using Android.Content;
using Android.Runtime;
using Android.Views;
using Android.Widget;
using Android.OS;

namespace Phoneword
{
    [Activity (Label = "Phone Word", MainLauncher = true)]
    public class MainActivity : Activity
    {
        protected override void OnCreate (Bundle bundle)
        {
            base.OnCreate (bundle);

            // Set our view from the "main" layout resource
            SetContentView (Resource.Layout.Main);

            // Our code will go here

        }
    }
}
```

20. Next, a reference is needed to the controls that were created in the layout file with the Android Designer. Add the following code inside the `OnCreate` method (after the call to `SetContentView`):

```
// Get our UI controls from the loaded layout
EditText phoneNumberText = FindViewById<EditText>(Resource.Id.PhoneNumberText);
TextView translatedPhoneWord = FindViewById<TextView>
(Resource.Id.TranslatedPhoneWord);
Button translateButton = FindViewById<Button>(Resource.Id.TranslateButton);
```

21. Add code that responds to user presses of the **Translate** button by adding the following code to the

OnCreate method (after the lines added in the last step):

```
// Add code to translate number
string translatedNumber = string.Empty;

translateButton.Click += (sender, e) =>
{
    // Translate user's alphanumeric phone number to numeric
    translatedNumber = PhonewordTranslator.ToNumber(phoneNumberText.Text);
    if (string.IsNullOrEmpty(translatedNumber))
    {
        translatedPhoneWord.Text = string.Empty;
    }
    else
    {
        translatedPhoneWord.Text = translatedNumber;
    }
};
```

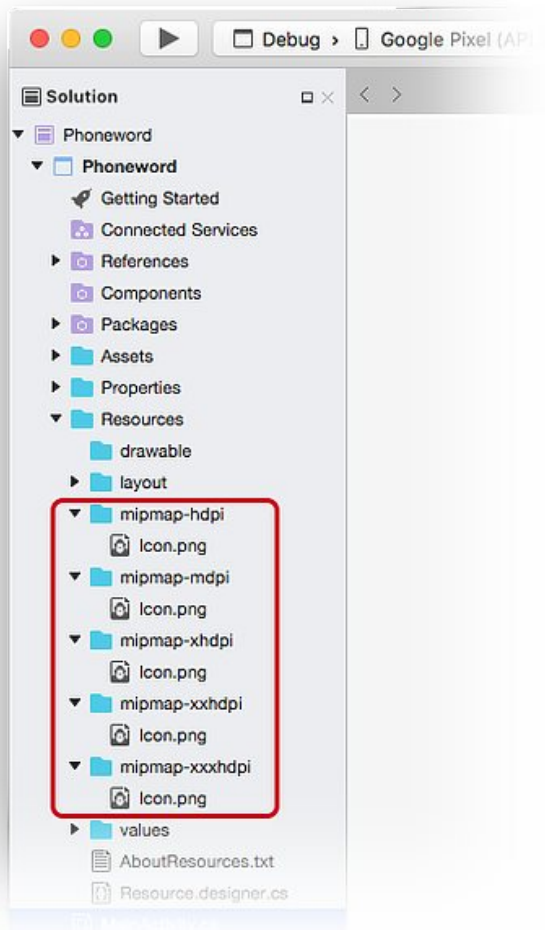
22. Save your work and build the application by selecting **Build > Build All** (or by pressing **⌘ + B**). If the application compiles, you will get a success message at the top of Visual Studio for Mac:

If there are errors, go through the previous steps and correct any mistakes until the application builds successfully. If you get a build error such as, *Resource does not exist in the current context*, verify that the namespace name in **MainActivity.cs** matches the project name (**Phoneword**) and then completely rebuild the solution. If you still get build errors, verify that you have installed the latest Xamarin.Android and Visual Studio for Mac updates.

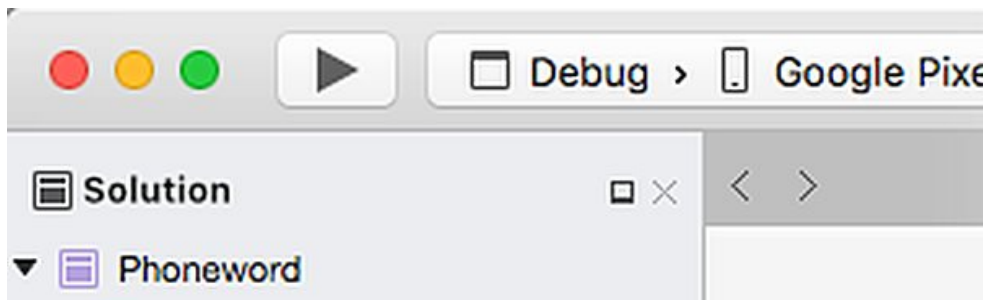
23. Now that you have a working application, it's time to add the finishing touches! Start by editing the **Label** for **MainActivity**. The **Label** is what Android displays at the top of the screen to let users know where they are in the application. At the top of the **MainActivity** class, change the **Label** to **Phone Word** as shown here:

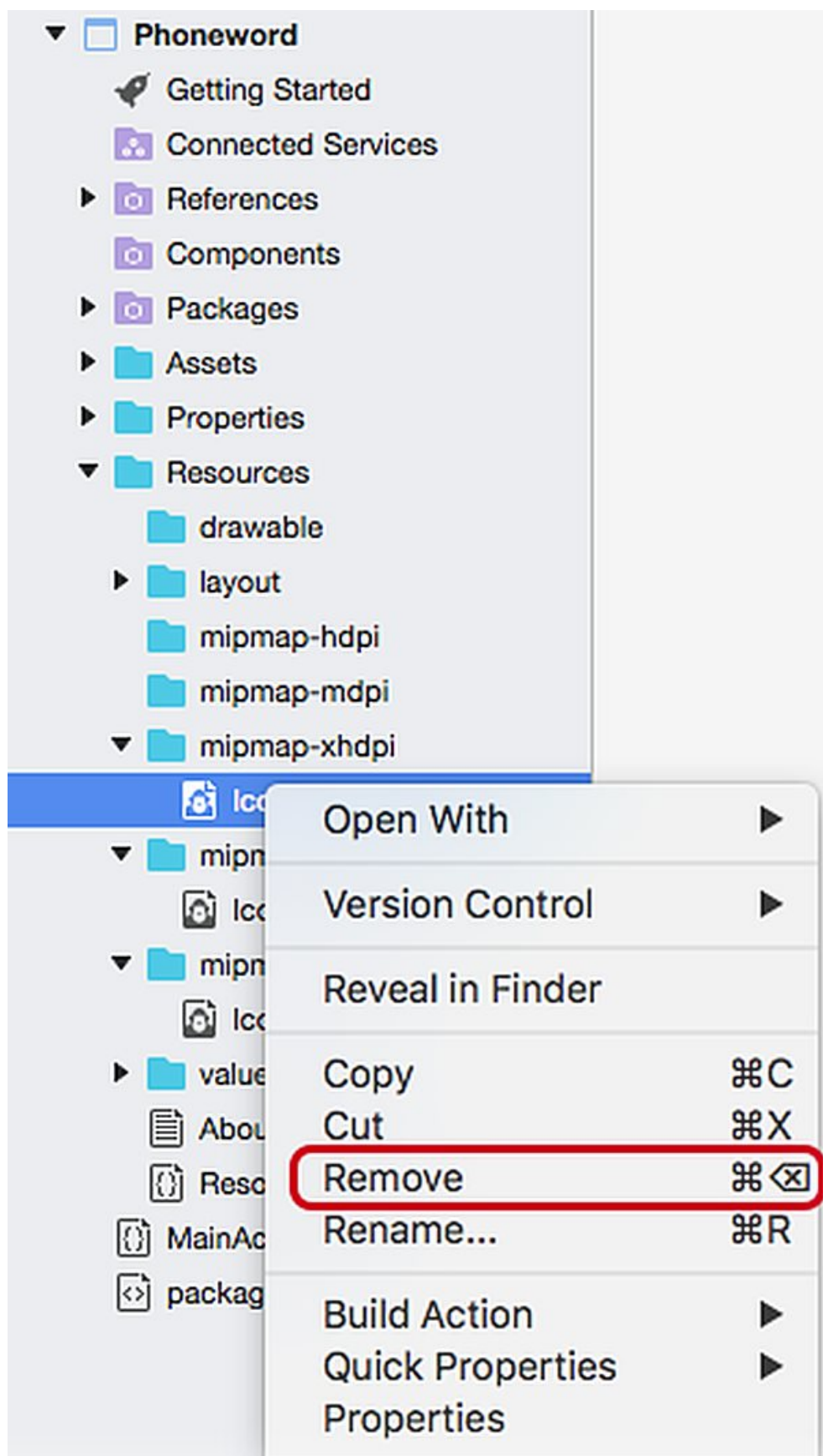
```
namespace Phoneword
{
    [Activity (Label = "Phone Word", MainLauncher = true)]
    public class MainActivity : Activity
    {
        ...
    }
}
```

24. Now it's time to set the application icon. By default, Visual Studio for Mac will provide a default icon for the project. Let's delete these files from the solution, and replace them with a different icon. Expand the **Resources** folder in the **Solution Pad**. Notice that there are five folders that are prefixed with **mipmap-**, and that each of these folders contains a single **Icon.png** file:



It is necessary to delete each of these icon files from the project. Right click on each of **Icon.png** files, and select **Remove** from the context menu:

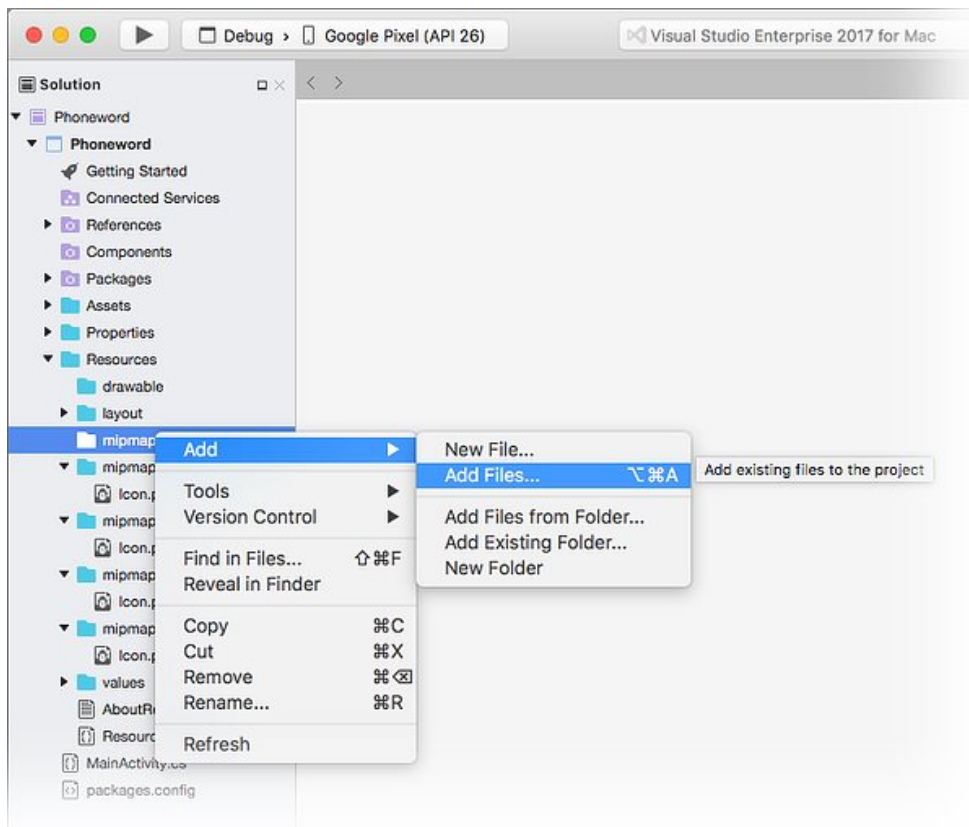




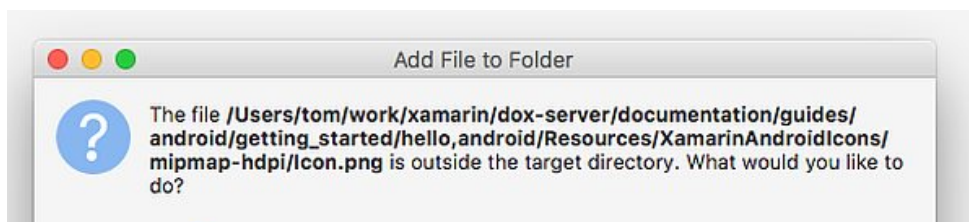


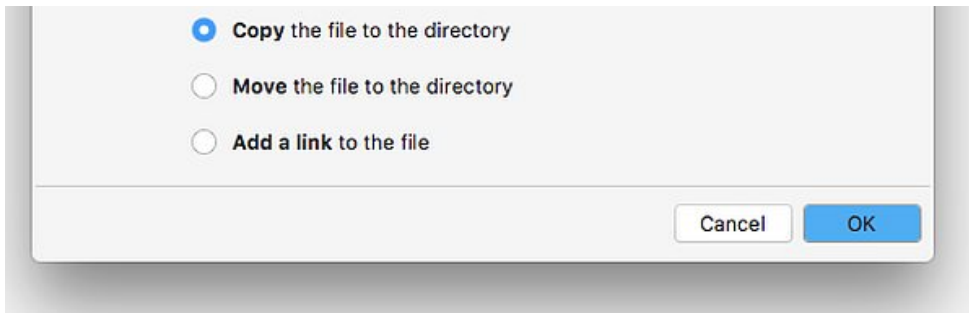
Click on the **Delete** button in the dialog.

25. Next, download and unzip [Xamarin App Icons set](#). This zip file holds the icons for the application. Each icon is visually identical but at different resolutions it renders correctly on different devices with different screen densities. The set of files must be copied into the Xamarin.Android project. In Visual Studio for Mac, in the **Solution Pad**, right-click the **mipmap-hdpi** folder and select **Add > Add Files:**

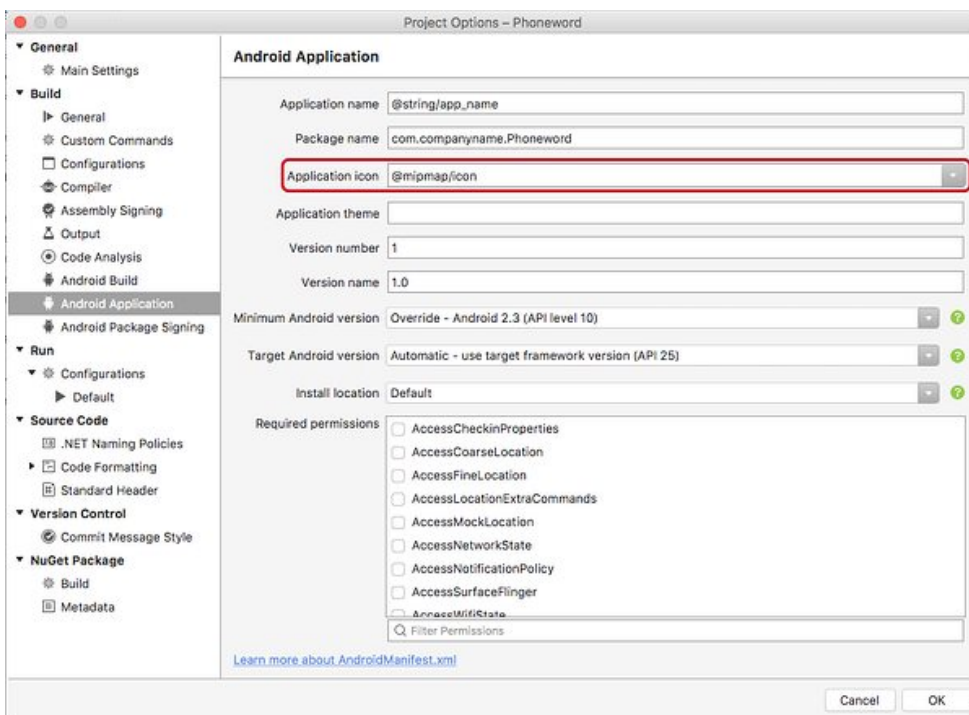


26. From the selection dialog, navigate to the unzipped Xamarin AdApp Icons directory and open the **mipmap-hdpi** folder. Select **Icon.png** and click **Open**.
27. In the **Add File to Folder** dialog box, select **Copy the file into the directory** and click **OK**:





28. Repeat these steps for each of the **mipmap**- folders until the contents of the **mipmap**- Xamarin App Icons folders are copied to their counterpart **mipmap**- folders in the **Phoneword** project.
29. After all the icons are copied to the Xamarin.Android project, open the **Project Options** dialog by right clicking on the project in the **Solution Pad**. Select **Build > Android Application** and select **@mipmap/icon** from the **Application icon** combo box:



30. Finally, test the application by running it on an Android device or emulator and translating a Phoneword:





[[/ide]]

Congratulations on completing your first Xamarin.Android application! Now it's time to dissect the tools and skills you have just learned. Next up is the [Hello, Android Deep Dive](#).